

HAFF PicklePulse

System Overview & Architecture

HAFF Leisure Club - Court rotation, queue, and live TV display

Document version: June 2026

Purpose: technical baseline for hosting, scaling, and cost planning.

Contents: architecture, data model, sync, APIs, deployment,
infrastructure profile, limitations, and recent fixes.

Document version: June 2026

Project: HAFF Leisure Club court rotation & live display system

Repository: Haff Leisure Club / `haff-picklepulse` v0.1.0

Executive Summary

HAFF PicklePulse is an **offline-first pickleball club operations app** built for HAFF Leisure Club (Cadiz City). It coordinates open-play nights across up to **three courts**, handling player check-in, queue/stack management, court assignment, live TV display, optional court reservations, community chat, and a local finance ledger.

The system is designed for **speed on game night**: admins tap **Finish** on courts rather than entering scores. Players see queue position and estimated wait time. A venue TV shows who is playing now, who is next, timers, and spoken announcements.

Primary surfaces:

Surface	Route	Who uses it
Landing	`/home`	Public marketing + login
Admin desk	`/admin`	Staff - full control
Player view	`/player`	Members - queue status, park/resume
Parking view	`/parking`	Players temporarily out of rotation
TV display	`/tv`, `/display`	Venue monitor - read-only board
Calendar	`/calendar`	Court reservation booking
Finance	`/finance`, `/payments`	Admin revenue ledger
Community	`/community`	Member chat

Business Workflow (Game Night)

Typical open-play flow

1. **Admin checks players in** from the roster (players cannot self check-in).
2. Admin builds **stacks** - groups of four players in queue order.
3. Admin **reserves a court** from the next stack (or assigns manually).
4. Match starts; **12-minute default timer** runs (configurable 5-45 min).
5. Admin taps **Finish** when the court is done - no score entry required.
6. Court frees; next stack can be assigned.
7. **TV display** mirrors courts, timers, next-up stacks, and TTS announcements.

Player parking

A checked-in player can **park** (sit out) without checking out. Parked players are excluded from stack, assignment, and reservation until they resume.

Overtime

When a match exceeds `matchDurationMinutes`, the timer shows negative time (overtime). TV may announce overtime via text-to-speech. Admin can also broadcast custom TV messages.

Reservations (separate from open play)

Members can request court bookings via the calendar. Admin approves, marks paid (manual - no payment gateway), and manages blackouts/allocations. Default court fee: **PHP 300/hour**.

User Roles & Permissions

Role	Database enum	Capabilities
Admin	`ADMIN`	Check-in/out, stack, courts, matches, timer adjust, TV broadcast, player CRUD, finance, reservation admin, testimonials moderation
Member	`MEMBER`	Login, self check-out, park/resume, own profile/kudos/reviews, reservation requests, community chat
TV browser	Any authenticated session	Read-only poll of live state; typically admin stays logged in on the TV

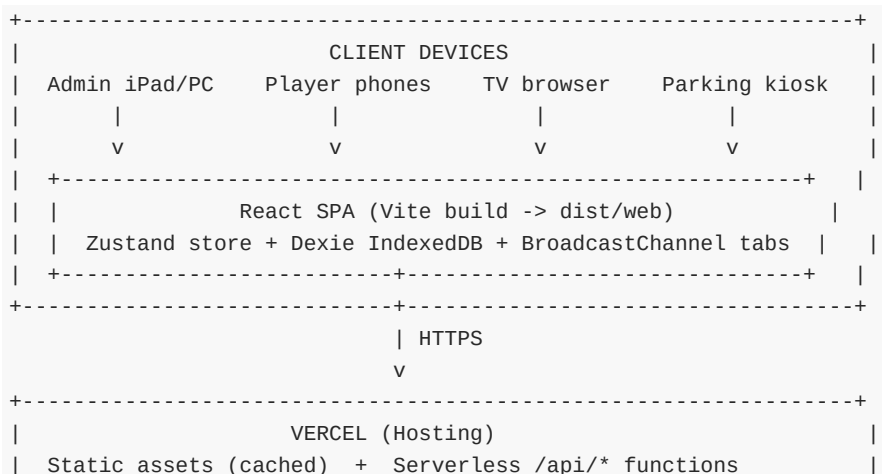
****Auth mechanism:**** Email + password -> HttpOnly cookie (`\_\_Secure-haff\_session`, 30 days). First account matching `INITIAL\_ADMIN\_EMAIL` becomes admin.

Technology Stack

Layer	Technology
Frontend	React 18, Vite 6, TypeScript, Tailwind CSS
UI components	Radix UI, Framer Motion, lucide-react
Client state	Zustand 5
Local persistence	Dexie 4 (IndexedDB) - offline-first
Sound / TTS	Howler.js + Web Speech API
PWA	vite-plugin-pwa (installable, service worker)
Production API	Vercel Serverless Functions (`/api/*.ts`)
Database	PostgreSQL via Prisma 5
Optional realtime	Ably (community chat push)
Analytics	Vercel Analytics
Hosting	Vercel (static SPA + serverless API)
Node runtime	22.x

****Not used in production:**** NestJS + Socket.IO skeleton in `apps/api` (local dev only). Socket.IO is explicitly disabled in `main.tsx` (`socket = null`).

Architecture Diagram



```

| Fast Origin Transfer meter applies to API responses |
+-----+-----+
|                                     |
|                                     | DATABASE_URL (pooler)
|                                     | v
+-----+-----+
|                                     |
| PostgreSQL (currently Supabase-hosted) |
| Prisma ORM - Users, Sessions, Matches, Reservations, Chat, etc. |
+-----+-----+

```

Optional: Ably <--> /api/realtime/token (community chat only)

Repository Structure

```

Haff Leisure Club/
+-- api/                # Production backend (Vercel serverless)
|   +-- club-state.ts   # * Core live sync (GET/POST)
|   +-- auth.ts         # Register, login, logout
|   +-- reservations.ts # Court booking workflow
|   +-- community.ts    # Chat
|   +-- sync.ts         # Legacy batch sync
|   +-- operations/events.ts # Idempotent operation log
|   +-- _auth.ts, _prisma.ts, _realtime.ts, _audit.ts
+-- apps/web/           # React frontend
|   +-- src/
|       +-- main.tsx     # All views + routing (~6900 lines)
|       +-- store/useClubStore.ts # Zustand (~2600 lines)
|       +-- lib/         # db, types, utils, sound, pricing
+-- prisma/schema.prisma # Database schema
+-- vercel.json          # Build + SPA rewrites
+-- dist/web/            # Production build output

```

Data Model

Live session state (hot path)

During active open play, most operational data lives in **Session.settings` JSON`**:

- `courts`, `matches`, `stackOrder`, `parkedPlayerIds`, `adminCheckedInIds`
- `playerProfiles`, `playerKudos`, `matchReviews`
- `reservations` (cached snapshot), `tvBroadcast`

This blob is read/written via `/api/club-state` on every sync.

Normalized Prisma entities

`Player`	Roster, skill, stats (games/days/streaks)
`Court`	Up to 3 courts, status, current match
`Session`	Game day record + `checkedInPlayerIds` + settings JSON
`Match`	Historical/normalized match records
`User` / `AuthSession`	Accounts and sessions
`CourtReservation`	Full booking workflow with payment status
`CourtAllocation` / `CourtBlackout`	Calendar availability rules
`ChatMessage` / `ChatReaction` / `ChatReport`	Community
`Testimonial` / `ImprovementReport`	Landing page content
`OperationEvent` / `SyncEvent`	Sync audit trails
`AuditLog`	Admin action log

Model

Note: Finance `Transaction` records exist in **IndexedDB only** - not synced to PostgreSQL.

API Endpoints

`/api/club-state` (core)

GET `?sessionId=&since=`

- Returns full session snapshot OR lightweight `{ unchanged: true, tvBroadcast }` when `since` matches `updatedAt`.
- Requires authenticated user (TV must stay logged in).

POST

- Admin: writes full operational state to `Session.settings`.
- Member: limited writes (check-out self, park, profile, reservations).
- `broadcastOnly: true`: TV announcement without full state merge.

`/api/auth`

Register, login, logout, change password, session revoke. Optional Cloudflare Turnstile on register (backend ready; UI not fully wired).

`/api/reservations`

Week calendar, request/cancel, admin approve/reject/mark-paid, settings, allocations, blackouts.

`/api/community`

Paginated chat, send/edit/delete, reactions, reports. Optional Ably push.

Other

- `/api/players`, `/api/courts` - public lists
- `/api/sync` - legacy admin batch sync
- `/api/operations/events` - idempotent offline queue replay
- `/api/testimonials`, `/api/feedback`, `/api/recap`
- `/api/realtime/token` - Ably token issuance

Sync & Multi-Device Architecture

How devices stay in sync

- Admin mutates** local Zustand + IndexedDB instantly (optimistic UI).
- `publishSharedState`** debounces (250ms) then **POST**s snapshot to server.
- Other devices** poll **GET /api/club-state** on an interval.
- Same-browser tabs** also use `BroadcastChannel` (`haff-club-state-v1`).

Polling intervals (current code)

TV display	15s	`allowUnchanged: true` after initial force load
Admin (no WebSocket)	15s	
Player parking view	20s	Skips when browser tab hidden
Pending sync count	30s	Shows offline queue depth

Bandwidth optimization (in code, deploy when unpaused)

Client

- `since=` conditional GET - unchanged sessions return ~100 bytes instead of full JSON.
- **Trimmed POST profiles** - only checked-in / stack / on-court / reserved players.
- **Exponential backoff** on API failures (`syncDegraded` banner on TV).

What is NOT active

- **Socket.IO / WebSockets** - disabled in production; all clients poll HTTP.
- **Supabase Realtime** - not used; app uses raw Postgres via Prisma only.

Offline behavior

- App **hydrates** from IndexedDB on load - works without network.
- Mutations queue locally; sync when online returns.
- Multi-device consistency requires network + auth for `/api/club-state`.

Key Features Detail

Courts & matches

- Max **3 courts** (client-enforced).
- Court statuses: Available, InUse, Paused, Maintenance, Reserved.
- Admin actions: reserve from stack, start, finish, clear, timer +/-30s, pause/unpause, reset warm-up.
- Match teams of 4; `vacant` / `reserved` placeholders in stack slots.

Stack & queue

- Ordered player IDs in groups of 4.
- Drag-and-drop in admin UI.
- Server normalizes stack against checked-in, non-parked players.

Check-in economics

- Admin check-in can auto-log **PHP 150 CheckInFee** transaction (local ledger).
- `AdminCheckedIn` tag tracks server-authorized check-ins.

TV display

- 3-column grid: Team A | timer | Team B (no scroll for 4 players).
- Elapsed/overtime timer runs **locally** via `requestAnimationFrame` - not per-tick network.
- TTS for assignments, overtime, admin broadcasts.
- Filters ghost/stub player names from announcements.

Finance view

- Admin-only; transactions in IndexedDB: CheckInFee, CourtReservation, SessionPass.
- Manual payment logging and void with reason.
- **No Stripe/GCash API integration** - manual admin workflow.

Community & landing

- Member chat with moderation.
- Testimonials on landing page (admin approval).
- Anonymous improvement feedback with rate limiting.

Authentication & Security

Item	Implementation
Password storage	bcrypt hash
Session token	32-byte random -> SHA-256 in DB

Cookie	HttpOnly, Secure, SameSite=Lax
API protection	`requireUser()` / `requireAdmin()` on routes
Turnstile	Optional on register (`TURNSTILE_SECRET_KEY`)
TV access	Requires valid session cookie on TV browser

Deployment Configuration

Vercel (`vercel.json`)

- Build: `npm run build:web` -> `dist/web`
- SPA rewrite: all non-API routes -> `index.html`
- API routes auto-deploy from `/api/`
- Static assets: 1-year immutable cache

Required environment variables

`DATABASE_URL`	Yes	PostgreSQL pooled connection
`DIRECT_URL`	Yes	Direct connection (migrations)
`INITIAL_ADMIN_EMAIL`	Recommended	First admin account
`TURNSTILE_SECRET_KEY`	Optional	Register CAPTCHA
`ABLY_API_KEY`	Optional	Chat realtime
`FEEDBACK_HASH_SECRET`	Optional	Feedback rate-limit salt

Domain

- Custom domain via Namecheap DNS -> Vercel (A record `76.76.21.21` or CNAME `cname.vercel-dns.com`).
- SSL automatic on Vercel after DNS propagation.

npm scripts

`npm run dev:web`	Local Vite dev :5173
`npm run dev:full`	Vercel dev (API + web together)
`npm run build:web`	Production build
`npm run network`	LAN-accessible dev (for offline venue testing)

Current Infrastructure Profile

Service	Role	Billing model	Known issue
Vercel Hobby	Host SPA + `/api` serverless	Free tier; **10 GB Fast Origin Transfer** / rolling 30-day window	Project **paused** at ~37 GB due to aggressive HTTP polling of full session JSON
Supabase	PostgreSQL host (`pooler.supabase.com`)	Free tier egress limits	Separate meter from Vercel; each `/api/club-state` hit reads DB
Namecheap	Domain registrar	Annual domain fee	DNS points to Vercel
Aby	Optional chat push	Free tier available	Graceful fallback to 10s poll
Cloudflare Turnstile	Optional bot protection	Free	Backend only

Why bandwidth spiked

- Socket.IO disabled -> **100% HTTP polling**.
- Previously TV polled every **1 second** with **full JSON** (~50-200 KB with 40 players).
- Multiple tabs/TVs multiply traffic.
- Vercel **Fast Origin Transfer** counts data from serverless functions to clients.

Fixes ready to deploy (reduce recurrence)

1. Conditional `?since=` GET (unchanged = tiny response).
2. TV/admin poll at 15-20s instead of 1-3s.
3. Smaller POST payloads (active players only).
4. Skip polling when browser tab is hidden.

Known Limitations

1. **Monolithic UI** - nearly all views in single `main.tsx` file.
2. **Live state in JSON blob** - not fully normalized during active play.
3. **No production WebSockets** - polling-only multi-device sync.
4. **Finance local-only** - transactions not in PostgreSQL.
5. **TV requires login** - session cookie must persist on TV browser.
6. **No payment gateway** - reservation fees tracked manually.
7. **Max 3 courts** - hardcoded.
8. **Turnstile UI** - backend ready, frontend doesn't send captcha token yet.

Recent Fixes (2026)

Area	Fix
Overtime	No emotional sound on Finish; timer hides when completed
Time sync	Admin timer adjust uses <code>publishSharedState({ force: true })</code>
Ghost players	Stricter merge rules; stub names show as "Queued" not "Player"
TV layout	3-column grid - 4 players visible without scroll
Check-in drift	<code>AdminCheckedIn</code> tag re-synced from server on refresh
Egress	<code>?since=</code> API + slower polls + trimmed profiles + backoff

Pricing Constants (App Logic)

Item	Amount
Court hourly fee	PHP 300
Check-in fee (optional auto-log)	PHP 150
Default match duration	12 minutes

Glossary

Term	Meaning
Stack	Queue of player groups (4 per court slot)
Park	Checked in but not in rotation

Open play	Walk-in game night mode (vs reserved courts)
club-state	The live session sync API and JSON snapshot
Fast Origin Transfer	Vercel meter for serverless -> client data transfer
IndexedDB / Dexie	Browser local database for offline-first
PWA	Installable progressive web app with service worker